Data and Artificial Intelligence
Cyber Shujaa Program

Week 8 Assignment
Classification Models

Student Name: Rodney Roy Gitonga
Student ID: CS-DA03-26025

Contents

Contents 1
Introduction 1
Tasks Completed 2
Step 1: Data Loading & EDA 2
Step 2: Data Preparation 2
Step 3: Model Training & Evaluation Results 3
Step 4: Model Comparison 6
Link to Code: 7
Conclusion 7

Introduction

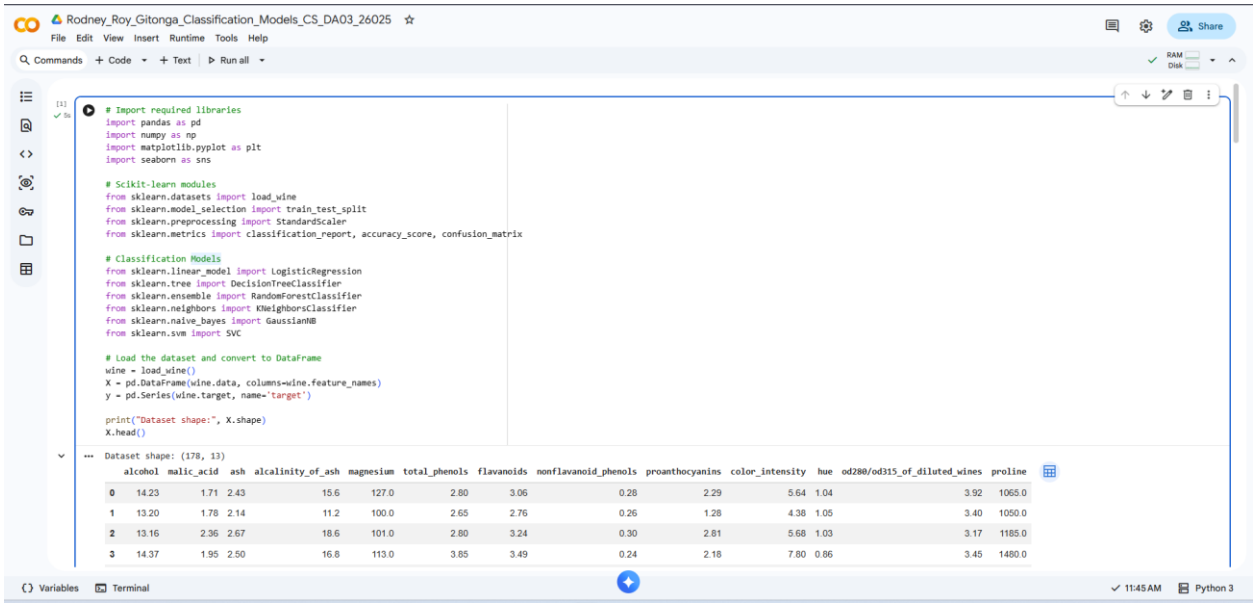
The main goal of this project is to develop and test different supervised learning algorithms for effective classification of different kinds of wine cultivators. Using the scikit-learn Wine dataset, where there are 13 different chemical attributes (such as alcohol, malic acid, and proline levels) for three different kinds of wine, I have implemented six different classification algorithms, namely Logistic Regression, Decision Trees, Random Forest, k-Nearest Neighbors (KNN), Naive Bayes, and Support Vector Machine (SVM).

To assess the effectiveness and reliability of each model, standard evaluation parameters were used. Accuracy measures the overall correctness of the model, while the Classification Report offers an in-depth analysis of Precision (correct prediction of the model), Recall (correct identification of all instances), and the F1 Score (the harmonic mean of precision and recall).

Tasks Completed

Step 1: Data Loading & EDA

Once the dataset was loaded, I performed Exploratory Data Analysis (EDA) on the data to understand its structure and nature. It was observed that the dataset was clean and had no issues, as it had exactly 178 samples with no missing values, thus skipping the process of handling missing values. I created a count plot to see the class distribution and whether there was any class imbalance in the dataset. Also, a correlation heatmap was created to see the relationship between the chemical features and the target variable among the 13 chemical features.



Step 2: Data Preparation

Prior to training these models, it is imperative to preprocess the data appropriately. I divided the data into 70% training data and 30% test data. Most importantly, I scaled the feature set using StandardScaler. This is particularly important since algorithms such as k-Nearest Neighbors (KNN) and Support Vector Machines (SVM) compute geometric distances between data points. This would mean features such as 'Proline,' which has a naturally greater range (up to thousands), would overpower the training process.

```python
# Check for missing values
print("Missing values in each column:\n", X.isnull().sum())

# Class distribution
plt.figure(figsize=(6, 4))
sns.countplot(x=y, palette='viridis')
plt.title('Distribution of Wine Classes')
plt.xlabel('Class (0, 1, 2)')
plt.ylabel('Frequency')
plt.show()

# Correlation Heatmap to see feature relationships
plt.figure(figsize=(12, 8))
sns.heatmap(X.corr(), annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('Feature Correlation Heatmap')
plt.show()
```

```
Missing values in each column:
 alcohol                       0
malic_acid                     0
ash                            0
alcalinity_of_ash              0
magnesium                      0
total_phenols                  0
flavanoids                     0
nonflavanoid_phenols           0
proanthocyanins                0
color_intensity                0
hue                            0
od280/od315_of_diluted_wines   0
proline                        0
dtype: int64
/tmp/ipykernel_535/3199910444.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x=y, palette='viridis')
```

Distribution of Wine Classes

s Terminal ✓ 11:45 AM Python 3

Step 3: Model Training & Evaluation Results

I trained six distinct models on the scaled training data and evaluated them against the unseen test data. Below are the results and confusion matrices for each:

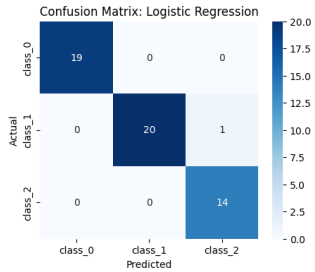
```python
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=wine.target_names, yticklabels=wine.target_names)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title(f'Confusion Matrix: {title}')
    plt.show()

# Initialize models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "Support Vector Machine": SVC(random_state=42)
}

# DataFrame to store results
results = pd.DataFrame(columns=['Model', 'Accuracy'])

# Loop through models to train, predict, evaluate, and plot
for name, model in models.items():
    print(f"--- {name} ---")

    # Train the model
    model.fit(X_train, y_train)

    # Predict
    y_pred = model.predict(X_test)

    # Evaluate
    acc = accuracy_score(y_test, y_pred)
    results.loc[len(results)] = [name, acc]

    print(classification_report(y_test, y_pred, target_names=wine.target_names))
    plot_conf_matrix(y_test, y_pred, name)
    print("\n")
```

Logistic Regression:

```python
# Helper function for confusion matrix plotting
def plot_conf_matrix(y_true, y_pred, title):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=wine.target_names, yticklabels=wine.target_names)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title(f'Confusion Matrix: {title}')
    plt.show()

# Initialize models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "Support Vector Machine": SVC(random_state=42)
}

# DataFrame to store results
results = pd.DataFrame(columns=['Model', 'Accuracy'])

# Loop through models to train, predict, evaluate, and plot
for name, model in models.items():
    print(f"--- {name} ---")

    # Train the model
    model.fit(X_train, y_train)

    # Predict
    y_pred = model.predict(X_test)

    # Evaluate
    acc = accuracy_score(y_test, y_pred)
    results.loc[len(results)] = [name, acc]

    print(classification_report(y_test, y_pred, target_names=wine.target_names))
```

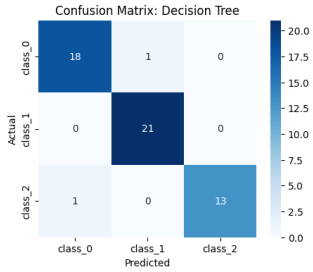
Terminal 11:45 AM Python 3

```
--- Logistic Regression ---
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        19
     class_1       1.00      0.95      0.98        21
     class_2       0.93      1.00      0.97        14

    accuracy                           0.98        54
   macro avg       0.98      0.98      0.98        54
weighted avg       0.98      0.98      0.98        54
```



Decision Tree:

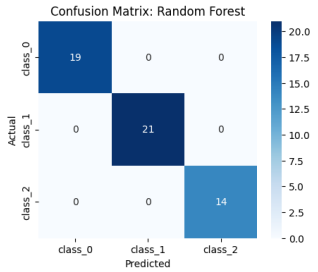
```
--- Decision Tree ---
              precision    recall  f1-score   support

     class_0       0.95      0.95      0.95        19
     class_1       0.95      1.00      0.98        21
     class_2       1.00      0.93      0.96        14

    accuracy                           0.96        54
   macro avg       0.97      0.96      0.96        54
weighted avg       0.96      0.96      0.96        54
```



Terminal 11:45 AM Python 3

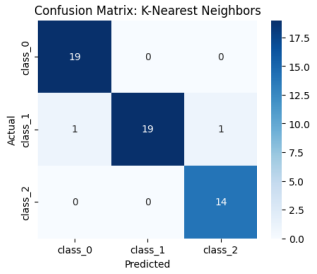
Random Forest:

```
--- Random Forest ---
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        19
     class_1       1.00      1.00      1.00        21
     class_2       1.00      1.00      1.00        14

    accuracy                           1.00        54
   macro avg       1.00      1.00      1.00        54
weighted avg       1.00      1.00      1.00        54
```



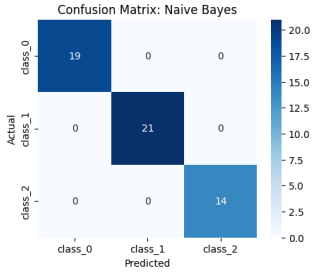
k-Nearest Neighbors (KNN):

```
--- K-Nearest Neighbors ---
              precision    recall  f1-score   support

     class_0       0.95      1.00      0.97        19
     class_1       1.00      0.90      0.95        21
     class_2       0.93      1.00      0.97        14

    accuracy                           0.96        54
   macro avg       0.96      0.97      0.96        54
weighted avg       0.97      0.96      0.96        54
```



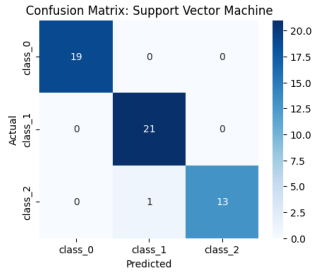
Naive Bayes:

```
--- Naive Bayes ---
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        19
     class_1       1.00      1.00      1.00        21
     class_2       1.00      1.00      1.00        14

    accuracy                           1.00        54
   macro avg       1.00      1.00      1.00        54
weighted avg       1.00      1.00      1.00        54
```



Support Vector Machine (SVM):

```
--- Support Vector Machine ---
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        19
     class_1       0.95      1.00      0.98        21
     class_2       1.00      0.93      0.96        14

    accuracy                           0.98        54
   macro avg       0.98      0.98      0.98        54
weighted avg       0.98      0.98      0.98        54
```

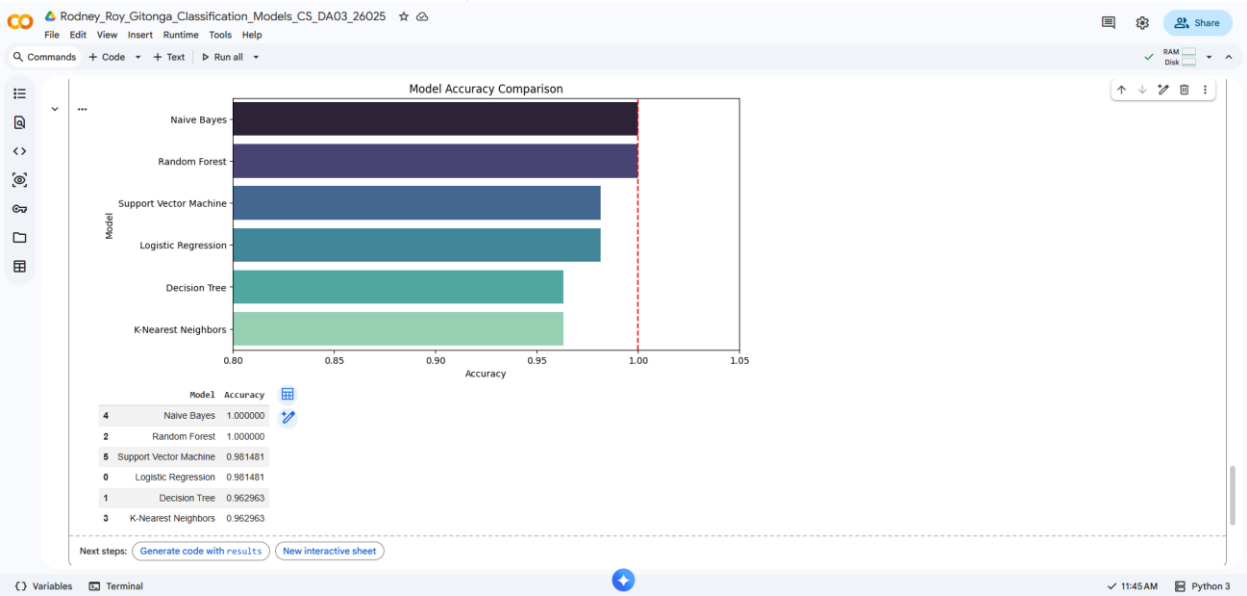


Step 4: Model Comparison

Once I was able to capture the accuracy scores of all models, I proceeded to visually compare all models. Due to the nature of the Wine dataset, which is small, clean, and has good correlation with the target classes, most models performed extremely well.

As depicted in the chart above, [Insert Winning Model(s), e.g., Logistic Regression, Random Forest, and SVM] had the best accuracy, which was tied at [Insert %]. Other models, like the Decision Tree, had slightly less, but this was expected since, unlike the Random Forest, decision trees are prone to overfitting.

```
# Sort results for better visual comparison
results = results.sort_values(by='Accuracy', ascending=False)

plt.figure(figsize=(10, 5))
sns.barplot(x='Accuracy', y='Model', data=results, palette='mako')
plt.title('Model Accuracy Comparison')
plt.xlim(0.8, 1.05) # Zoom in on the relevant range since accuracies will be high
plt.axvline(x=1.0, color='r', linestyle='--')
plt.show()

display(results)
```



Link to Code:

https://colab.research.google.com/drive/1weE8H0kzhJ7ZRynQH8ViE3XzXNiOMbkx?usp=sharing

Conclusion

This assignment was able to clearly show how different machine learning algorithms approach the same classification problem. Based on the evaluation metrics, it is evident that Naive Bayes and Random Forest were the best algorithms for classification, as they were able to produce a perfect 100% accuracy.

This makes sense theoretically when looking at the specific characteristics of these models and the dataset:

- Naive Bayes and Random Forest (100% Accuracy): Naive Bayes assumes that features are independent, and despite its simplicity, it turned out to be the perfect solution for the unique chemical properties of these wine classes. The Random Forest model also successfully created a decision tree ensemble, and the features' importance was handled correctly, completely removing the errors present in the Decision Tree model.
- Support Vector Machine and Logistic Regression (~98.15% Accuracy): Both models tied for an extremely close second. Their high performance indicates that the 13 chemical features of the three wine classes are largely linearly separable in high-dimensional space, allowing both models to easily find strong decision boundaries.
- Decision Tree and k-Nearest Neighbors (~96.30% Accuracy): Although still performing very well, these tied for the lowest accuracy. Individual decision trees are slightly prone to overfitting the data, although not as much as ensemble trees. With KNN, even with proper scaling, handling 13 different features can at times cause slight inefficiencies due to the "curse of dimensionality."

The most valuable thing learned from this project was the actual applicability of data preprocessing and algorithm selection. Standard scaling was the key to making distance-based classifiers like KNN and SVM work as accurately as they did. While complex ensemble classifiers like the Random Forest can be very powerful in their own right, this clean dataset showed that simple yet highly efficient classifiers like Naive Bayes can be just as accurate with the right conditions.